

SHL Assessment Recommendation System

Approach Document

Author: Mohammad Inayat Hussain

Page 1: Problem, Architecture and Data Pipeline

Problem

The goal was to build a system where someone can type what kind of test they need like "I need a coding test for Java" and get the best matching SHL assessments back. The system had to be a real API returning JSON, and it also needed to generate a CSV file predicting answers for a set of test questions.

How it works

The way the system finds the right assessments is pretty straightforward. First it takes the user's search query and cleans it up. Then it looks for specific rules in the text, like if the user asked for a test under 30 minutes, or a specific type like a personality test.

After that, the system scores every SHL assessment against the user's search to find the best matches. It scores them based on three things:

First is general meaning. It uses an embedding model called all-MiniLM-L6-v2 to understand what the search means and compares it to all the assessments. This is worth half the score.

Second is exact word matches. Sometimes meaning isn't enough. If someone searches for Java, they want a Java test, not a generic coding test. So the system gets extra points for matching exact words.

Third is title matching. If a user searches for something that exactly matches the title of an SHL test, that test gets a big boost to the top of the list.

Getting the data

To get the list of SHL assessments, I wrote a Python script using BeautifulSoup that went through the SHL website. It went to every single product page and pulled out the description, how long the test takes, what kind of test it is, and whether it can be taken remotely. Sometimes the SHL website would give an error, so the script had to wait and try again when that happened.

Once I had all the data, I had to clean it up. I removed duplicate entries, fixed formatting issues so the computer could read it better, and combined all the important information into one big block of text for each assessment so the search engine could understand it. I ended up with 389 unique assessments.

Tools used

For the API I used FastAPI. To understand the text I used a sentence transformer model from Hugging Face. The search logic was built with standard Python math libraries like NumPy. I built a quick web interface using Streamlit, and deployed the whole thing on Hugging Face Spaces and Streamlit Cloud.

Page 2: Evaluation, Decisions and Security

How I tested it

To see if the system was actually good, I tested it against the 10 example queries SHL provided where we already knew the right answers.

At first, I just used the embedding model by itself. It was decent, but it missed a lot of specific technical tests.

Then I tried adding exact name matching, but some weird characters in the scraped data broke it and made the score worse. Once I fixed the data cleaning, the score jumped up significantly.

The biggest breakthrough was adding a step before the search where I map common words to SHL's specific vocabulary. For example, if a user asks for a behavioral test, the system knows to look for the OPQ personality questionnaire. Combining that with the three-part scoring system gave the best results.

Why I built it this way

I chose to combine meaning-based search with exact word matching because neither one works perfectly on its own. Meaning is good for broad searches, but terrible for specific software languages.

I decided not to use a big complex vector database like Pinecone because there are only 389 assessments. A simple math array in Python can search through 389 items almost instantly, so a database would just add unnecessary complication.

I also made sure the system works perfectly even if the Gemini AI expansion fails. I didn't want the API to break just because an external service went down.

Keeping it safe

The API is completely public so it had to be secure. Every search query is sanitized to remove any malicious code before the system processes it.

I also added protection to make sure users can't trick the API into accessing hidden internal servers, which is a common vulnerability. The API limits how many times someone can search per minute to prevent abuse, and if something does crash, it hides the error details so attackers can't learn anything about the server.

What could be better

Right now, if someone pastes a LinkedIn job link, the system just tries to guess keywords from the URL because LinkedIn blocks bots from reading the actual job page. It works okay, but actually reading the job description would be better.

Also, if a user just types one very generic word like "test", the system doesn't have much to go on, so the results aren't as relevant.

In the future, it would be great to actually train the embedding model specifically on SHL's data, instead of using a generic one. That would make the understanding of the specific HR terminology much sharper.